

METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation

Summary Team: Aaron Bartlett (aaronbar), Brandon Chao (bchao), Uygur Tepe (tepe)

Problem

RAG systems have a quality delay tradeoff:

- Retrieving too many chunks for simple queries adds unnecessary wait time
- No solution for tuning configuration to optimize for these factors with scheduling in mind

These are important as they can greatly impact user experience as well as operational costs.

METIS introduces a system to configure a system on a per-query basis to optimize the quality-delay tradeoff.

Related Works

Previous works have tried to solve the quality and delay problems in isolation. For instance, previous works looked to identify better query scheduling to reduce response delays at the expense of static allocations and configurations. Inversely, prior works have looked at improving quality by dynamically adjusting RAG configurations at the expense of response delays.

READ: Scheduling and resource allocation optimizations to improve latency

“Fast inference for augmented large language models” (Shahout et al)

READ: RAG configuration and context choices to optimize for quality of output

“Mba-rag: a bandit approach for adaptive retrieval-augmented generation through question complexity” (Tang et al)

READ: Overview of RAG applications in LLMs

“Retrieval-augmented generation for large language models: survey” (Gao et al)

KV caches can also be highly utilized (in theory) with RAG. But because more contexts are needed, storing all the information can become prohibitively expensive.

Proposed Solution

RAG is composed of 2 main stages:

- Retrieval: Retrieve relevant chunks of information from the database using embeddings (vector database)
- Synthesis: Combining chunks with the RAG query to form LLM calls

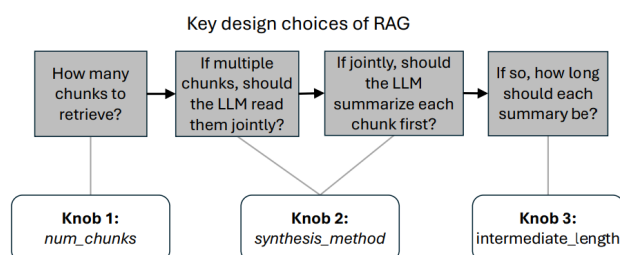
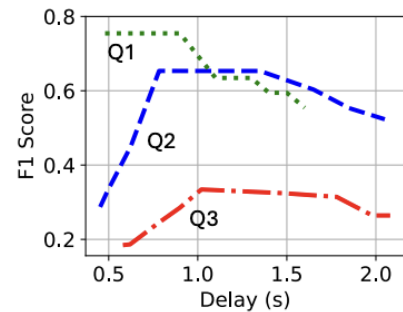


Figure 2. The configuration knobs adapted by METIS are derived from key design choices of RAG systems.

Using this information, the authors allow for the configuration of 3 variables based on the RAG stages: number of chunks, synthesis method, and intermediate length. Where the *number of chunks* correlates with the response delay, the *synthesis method* determines how to query chunks, which impacts both response delay as well as quality, and the *intermediate length* determines the size of the summary response.

Retrieved Chunks

With more complex requests, there may be a need to retrieve more data. But doing so blindly can hide or mask information that will provide the most value. As seen in the figure below, increasing the number of chunks can negatively affect output quality for less complex queries.



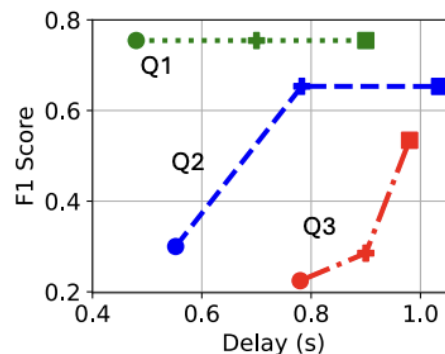
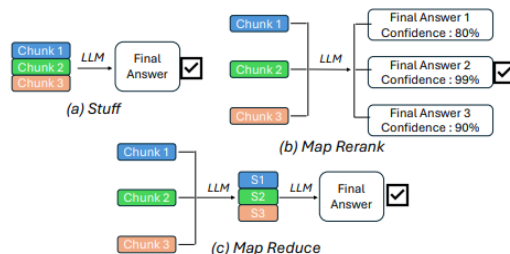
(b) Change: Increase # of chunks from 1 to 35 with stuff (left to right)

Synthesis Method

The authors outline 3 different methodologies: stuff, map rerank, and map reduce. The three algorithms are diagrammed below and produce different qualities of response.

- **map_rerank** processes chunks individually and chooses the best output. This minimizes latency but struggles when multiple sources of information are necessary
- **stuff** processes all chunks into a single summary for the LLM. This combines many sources into one but can lose details of each if specific context is necessary
- **map_reduce** produces summaries for each chunk and provides them all to the LLM. This preserves the most information and is necessary for the most complex queries.

As seen in the results figure below, **map_reduce** comes with longer latency but improved results for more complex queries. However, for simpler queries, any algorithm produces quality results.

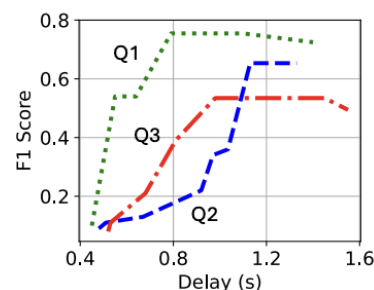


(a) Change: synthesis method from map_rerank (circle), stuff (plus) and map_reduce (square)

Figure 3. Illustration of different RAG synthesis methods which have various LLM reasoning capabilities.

Intermediate Output Length

The **map_reduce** algorithm combines retrieved chunks into an intermediate phase. This coalesces several sources into a single input. Increasing the length of the intermediate representation allows for more information to be preserved. However, as seen in the figure below, the longer summary may not be necessary for simpler queries and it comes at the cost of added latency to the already extended **map_reduce** algorithm.



(c) Change: Increase intermediate length from 1 to 100 with map_reduce (left to right)

The complexity of RAG requests requires different levels of inter-chunk reasoning and, therefore, varying numbers of context chunks. METIS tunes these parameters for each query. These tuned configurations are chosen from a database of profiled options. METIS is divided into two main components: the pruning configuration space and the RAG scheduler.

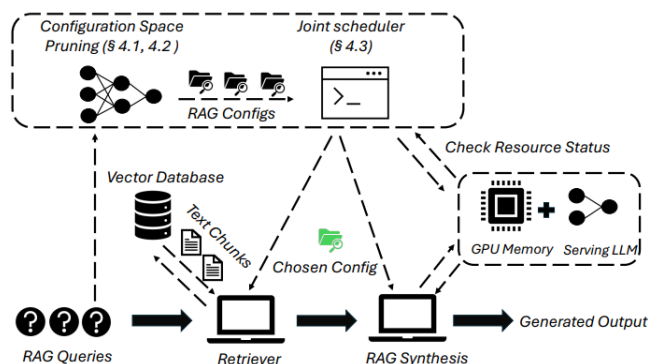


Figure 6. METIS consists of a RAG controller which performs configuration space pruning and joint scheduling.

METIS can be broken down into 3 stages: a query profiler, which will estimate with coarse granularity, followed by the pruned configuration space, which will identify a subregion in which the optimal parameters will be found, and lastly a joint scheduler, which will choose the exact configuration to use for a specific query.

Query Profiler

A tuned Profiler LLM is used to answer four questions about a query and return specific evaluations of each. The query profiler LLM is prompted with the follow questions:

- “Estimate the query complexity” - Binary High/Low
- “Do we need joint reasoning” - Boolean Yes/No describing whether multiple chunks of context will be required to answer the question
- “How many pieces of information required? - Integer 1 to 10

- “How much can we summarize?”- If responses are long, how long should the summarized portion be in the range of 30-200

These parameters are condensed used to map to the three knobs mentioned above through a simple rule-based algorithm found below.

Algorithm 1: Rule based mapping algorithm

Input: *Query complexity, Joint reasoning required*
Input: *Pieces of information , Summarization length range*
Result: *synthesis_method, num_chunks, intermediate_length*

```

1 if Joint reasoning required == “no” then
2   | synthesis_method = map_rerank
3 else
4   | if Query complexity == “low” then
5     | synthesis_method = stuff
6   | else
7     | synthesis_method = stuff, map_reduce
8 num_chunks = [Pieces of information , 3× Pieces of information]
9 intermediate_length_range = Summarization length range

```

In the last stage, which will conform to a set configuration, the authors also consider GPU memory constraints. In the event that none of the possible configurations will fit into the GPU memory space, METIS will fall back to a cheaper, less detailed configuration.

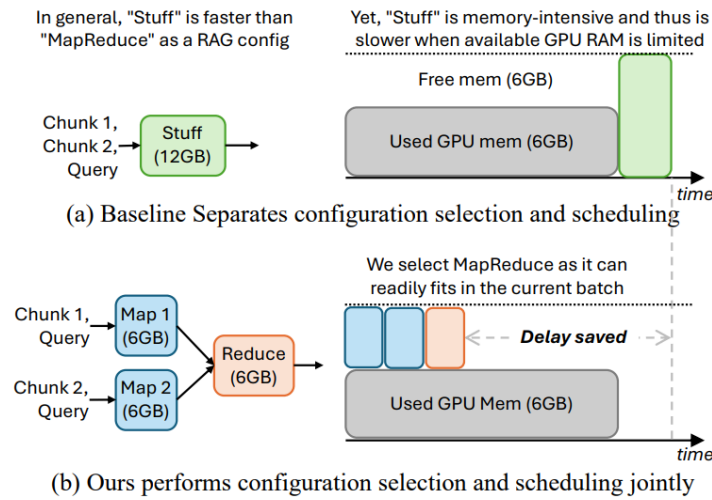


Figure 8. METIS joint schedules RAG configurations with available GPU memory (chosen example - map_reduce)

Evaluation

METIS was able to achieve better results (F1 Score) with much less latency in comparison to Parrot, AdaptiveRAG, and vLLM. Parrot represents a fixed RAG configuration; AdaptiveRAG is the earlier-mentioned selection of an optimal answer configuration at the cost of performance. vLLM is the baseline.

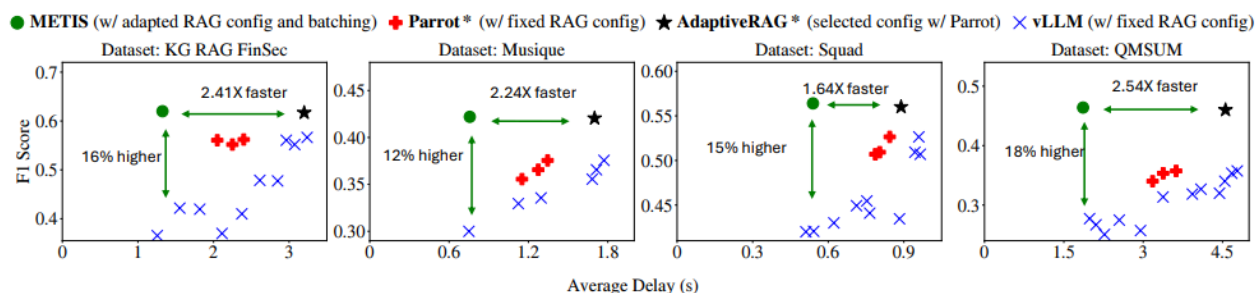


Figure 10. METIS achieves 1.64 – 2.54× lower delay compared to both best fixed configuration baselines and quality-optimized RAG configuration without sacrificing generation quality.

Limitations

The configurations from the database come from profiling. But due to the exponential search space, not all optimal solutions are included. Furthermore, online profiling (while queries are run) could result in better performance but results in higher computational costs. Even more agentic and reasoning RAG would need even more adaptable profiling and configuration to prove meaningful.

From the presenters:

- Rule-based mapping is difficult to extend
- Results, evaluation, and ablation studies are only focused on one benchmark AdaptiveRAG

Future Directions

Agentic workflows are a relatively new field of study that will likely use a deep chain of thought for RAG. METIS offers opportunities to profile these use cases and workloads to see whether sub-queries generated by the chain of thought are being efficiently utilized. Additionally, multimodal data retrieval is a logical extension of this work as more and more work pushes for data beyond text.

Audience Q&A

- Can stored chunks be summarized offline and stored as summaries?
 - The length of summary can vary depending on the query so it may not be optimal to store a single summary for each text
- If the task were extended past simple Q&A would the Profiler LLM need to be retrained?
 - The Profiler LLM is a simple LLM, they use LLAMA3 but have shown that the profiler is stable with other LLMs used without fine tuning. It would not need to be changed or retrained for different prompts
- METIS chooses the best configuration that fits into the GPU but the rule-base mapping algorithm doesn't consider memory available? How does METIS map to the next-best configuration if memory is unavailable?
 - not asked
- How does the profiler decide on downstream configurations and possible failure points?
 - The scheduler is reliable with 90% confidence in profiling. If confidence is lower, then the query profiler is not used
- Uncertainty propagation(bias/exploding variance) in RAG systems is possible. How does METIS handle uncertainty between retrieval and synthesis?
 - The paper does not include external reranking of chunks by similarity like other papers do. This may reduce uncertainty downstream by prioritizing the best chunks. A separate LLM can be trained to rerank chunks by priority based on a given query.
 - "This similarity reranking should be inside of the vector database and not need to be external." Jae-Won

RUMMY: Fast Vector Query Processing for Large Datasets Beyond GPU Memory with Reordered Pipelining

Background Info

- Vector query uses ML to translate the similarity of text, image and audio to a high-dimensional vector space, but GPU memory limitations make billion-scale datasets difficult to handle
- These vector databases are essential for modern AI systems, such as retrieval-augmented generation (RAG), recommendation systems, and semantic search.
- However, GPU memory limitations make billion-scale datasets (hundreds of GB) difficult to process efficiently.
- Training large models is no longer the only system challenge; processing and serving large datasets on limited GPU memory has become equally critical.

Related Work

- Inverted File Index (**IVF**) is a structure used by the Approximate Nearest Neighbors (**ANN**) algorithm supported by RUMMY. IVF minimizes the number of vectors searched and allows the search to be partitioned on a GPU without excess memory footprint
 - **READ:** IVF applied to similar search on GPUs
 - *“Billion-scale similarity search with GPUs” (Johnson et al)*
 - **READ:** Memory minimization for on-device ANN search
 - *“SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search” (Chen et al)*
- Retrieval-Augmented Generation (**RAG**) is a technique that allows LLMs or other models to improve their responses by querying outside data from a database or the internet to inform their responses. This technique provides serving problems and vector querying is typically used to optimize RAG processes.
 - **READ:** Applications of RAG to “Retrieval-augmented generation for knowledge-intensive nlp tasks.” (Lewis et al)

Problem

Most vector query processing today is still CPU-based, because GPUs (with 40–80 GB of memory) cannot store large datasets like TEXT1B (748 GB).

This limits the scalability and efficiency of AI systems that rely on fast similarity searches.

RUMMY addresses this by overlapping data transmission (host → GPU) with GPU computation, thereby utilizing GPU resources efficiently even when the dataset exceeds GPU memory.

System Challenges:

1. Limited GPU memory
 - a. Existing GPU-based systems must load the entire dataset into GPU memory.
 - b. Billion-scale datasets (100s of GB) exceed GPU capacity
2. Sequential transmission and computation
 - a. Naive host–GPU swapping (rotating data parts) causes long idle GPU time
3. Inefficient CUDA unified memory
 - a. General-purpose memory swapping leads to massive GPU page faults and poor overlap between transfer and compute
4. Underutilized GPU SMs
 - a. Small or unbalanced batches cause spatial (idle SMs) and temporal (straggler) underutilization
5. Redundant data transfers across queries
 - a. Multiple queries often require overlapping clusters, leading to repeated transmission of the same data

Importance

- Vector databases are becoming the long-term memory of large language models.
- Bottlenecks in vector query processing directly affect response latency, scalability, and cost-efficiency of AI systems.
- GPU-based processing can be orders of magnitude faster, but the limited GPU memory prevents straightforward use at scale.
- Overcoming this bottleneck enables high-performance, cost-effective AI applications using existing hardware.

Related Works

State of Related Works

- GPU-based systems: Require the entire dataset to be loaded into GPU memory.
 - Some approaches rotate data between the host and the GPU, but this is sequential and leaves the GPUs idle.
- CUDA Unified Memory: Automatically swaps memory between host and GPU, but it is query-unaware and causes massive GPU page faults, reducing performance.
- CPU-based systems: Can handle large datasets, but are much slower and less cost-effective.

Key Limitations

- Poor overlap between transmission and computation.
- Redundant data transfers across queries.
- Inefficient GPU utilization (idle or straggling streaming multiprocessors, SMs).
- No mechanism to intelligently schedule or pipeline vector queries.

Proposed Solution and Key Insight

RUMMY

- RUMMY is the first GPU-accelerated vector query processing system that:
 - Supports datasets beyond GPU memory capacity
 - Achieves higher throughput and cost-efficiency than both CPU and prior GPU systems
- Core innovation: Reordered Pipelining Technique; efficiently overlaps host-GPU transmission and GPU computation.

Key Insights

Vector query workloads are **independent and reorderable**. Unlike deep learning model layers (which must execute in sequence), vector queries can be **dynamically reordered** to pipeline transmission and computation optimally at runtime.

Main techniques:

1. Cluster-based Retrofitting – eliminate redundant data transmission
 - a. Observes that each cluster (data subset) can be processed independently
 - b. Retrofitting rearranges query plans to ensure each cluster is transmitted only once per batch
 - c. Handles:
 - i. Intra-batch reuse: reuse shared clusters among queries
 - ii. Inter-batch reuse: prioritize clusters already in GPU memory for next batch

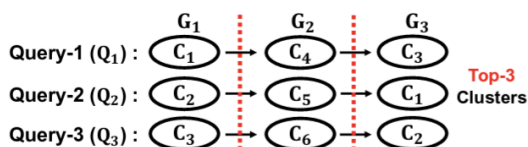


Figure 1: Query plan for a batch of three queries.

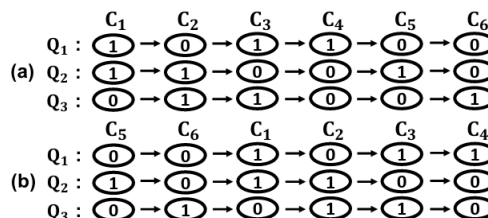


Figure 5: Retrofit the vector query plan.

Figure 5 depicts a retrofitted query plan based on Figure 1

2. Dynamic Kernel Padding with Cluster Balancing – maximize GPU utilization
 - a. Cluster balancing (done offline)
 - i. Splits clusters into equal-sized, balanced subclusters to remove straggler effects
 - ii. Equal work per GPU thread, which maximizes temporal GPU utilization
 - b. Dynamic kernel padding (done online)
 - i. Split thread blocks further at runtime to fill all GPU SMs, achieving 100% spatial utilization
 - ii. Pads underutilized SMs with lightweight blocks to achieve 100% GPU occupancy
 - c. Key Impact: All SMs busy, no stragglers -> significantly higher GPU utilization
3. Query-aware Reordering and Grouping – optimally overlap transmission and computation (Pipelining Scheduler)
 - a. Profiler: predicts transmission and computation for each group
 - i. Transmission time = $a \times p \times m + b$, where a and b are parameters fitted offline, and p is the size of balanced clusters
 - ii. Computation time estimated via offline profiling map
 - b. Reordering
 - i. Move clusters with zero transmission time (cached) or large computation time to the front
 - ii. Moves the clusters with zero transmission time to the front, solving the inter-batch problem
 - c. Grouping
 - i. Finds the optimal group granularity: balancing overhead vs. grouping all clusters (no overlap between transmission and computation)
 - ii. Prunes search space using lower-bound heuristics
 - d. Key Impact: maximizes overlap between transmission and computation with negligible runtime-overhead (<0.5%)

Evaluation and Results:

Experimental Setup:

- Datasets: SIFT1B, DEEP1B, TEXT1B (billion-scale)
- Hardware: NVIDIA A100 GPU (40 GB) vs. 64 vCPUs (CPU baseline)
- Batch sizes tested: small (8) and large (2048)

Performance:

- Up to 135× faster than IVF-GPU using CUDA unified memory
- Up to 23.1× faster and 37.7× more cost-effective than a state-of-the-art CPU solution
- Near-optimal pipeline efficiency, close to the theoretical lower bound

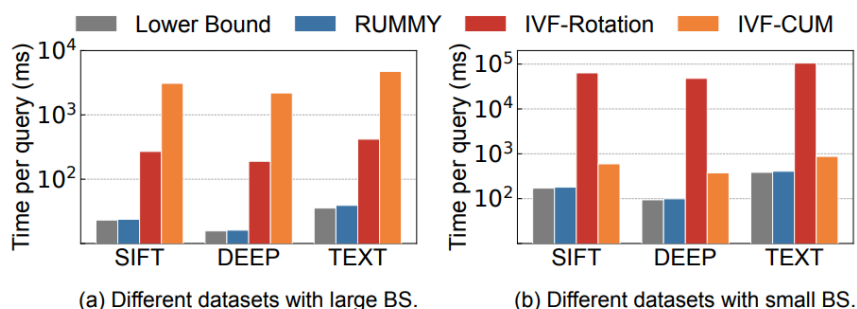


Figure 9: Overall performance on GPUs.

Limitations and Drawbacks

- Dependent on IVF indexing: RUMMY assumes cluster-based independence, making it less applicable to graph-based ANN indexes (e.g., HNSW).
- Offline preprocessing overhead: Cluster balancing and profiling add setup cost; less suited for rapidly changing datasets.
- Single-GPU scope: Does not address multi-GPU or distributed scaling.
- Static optimization: Runtime reordering is heuristic; does not adaptively learn from workload dynamics.

Potential Future Research Directions

- Multi-GPU and distributed pipelining: Extend reordered pipelining across multiple GPUs and networked systems.
- Adaptive runtime learning: Use reinforcement learning to adapt scheduling policies based on query load.
- Generalization to other ANN structures: Apply RUMMY's concepts to graph-based or hybrid indexes.
- Integration with emerging GPU interconnects: Combine with NVLink, CXL, or GPUDirect Storage for larger virtual memory pools.
- Energy- and cost-aware optimization: Extend pipelining decisions to minimize power and cloud cost.

Audience Q&A

- Are the vector databases static or changing
 - It can be changed based on the data corpus. For example, a company database may be updated annually to reflect new information.
- (Figure 4) Does the paper discuss integration with hierarchical storage?
 - The paper is limited to a single GPU system using just PCIe and does not discuss multi-GPU or hierarchical systems. If the paper were to run on multiple GPUs, it would simply duplicate its system and shard into multiple non-overlapping batches.
- Does the problem this is trying to address get minimized if GPU memory increases?
 - Usually, the max GPU memory is 96GB. This is better, but it does not remove the need for RUMMY. In fact, this system would be improved by larger GPU memory
- How easy would RUMMY be to optimize across different vector search methodologies(paper looks mainly at ANN)?
 - ANN is not precise, but approximates the closest vector since it would be too expensive to find the exact optimal solution
- Does the scheduler affect fairness between longer and shorter batches?
 - Queries are still technically FIFO but the combined batches are reordered.
- Could RL be used to schedule the batches (in place of the current DP algorithm)
 - RL would overcomplicate and may not provide sufficient returns

Class Discussion

What about memory?

- For given tasks, agents will have isolated memory logically and will not need any physical separation between different vectors in the database.
 - This can result in LLMs getting unrelated topics and vectors being conflated and brought up during an unrelated conversation. It may be useful to have a separation of memory and data so that the user can see what the LLM is pulling from.
 - In a general context, people will organize their prompts, data, and discussions differently so it may not be easy to select the correct data to be used for each user specifically.
- What about short-term versus long-term memory usage
 - Per query needed context versus documents that need to be stored in the long run

GPT5 Rant

- Jae-Won cancelled his GPT-Pro, thinks GPT5 is getting dumber and getting more contexts confused. Long context prompts and queries are creating more issues where the prompted optimization or question can get lost within the long context.

What is the exciting version of the vector query storage problem?

- This seems useful but not exciting or a major breakthrough -mosharaf
- How can you cache chunks from each RAG request to re-use these
 - Could you use LM Cache to store caches of chunks based on RAG requests?
- Agentic RAG workflows:
 - Similar RAG requests get set with some chunks being reused. Takes advantage of temporal locality.

Integration with Tools for Agents

- What else should be integrated beyond just these tools? The main issue is that agents do not have full control over tool usage (third party). Is this enough? Should there be different workflows for users?
- Multi-modal RAG will likely be similar but much larger data and come with possible memory issues
- Would it be possible for multiple agents interacting with the same database to result in races or issues with conflicting access. Could this be introduced? There are not many writes to these contexts so it is unlikely that multiple changes will be attempted at the same time.
 - Could see multiple writer, multiple reader paradigm in the future
- There may need to be prompt injection to add structure to prompts which are currently flat. Each token being semantically equivalent prevents the most critical parts of the prompt from being the most emphasized.

Q1. Since the profiler determines all downstream configurations, isn't it a single point of failure? Are there mechanisms to handle that?

A: The profiler is only used when the inference model is at least **90% confident**. If confidence falls below that threshold, the system **doesn't create or use a profile**, avoiding incorrect configuration propagation. However, this does mean that if the profiler fails, there's currently no fallback strategy—just rejection of uncertain profiles.

Q2. In the first paper, how does Mendis handle uncertainty propagation between retrieval and synthesis, especially when exploring large configuration spaces?

A: The paper doesn't directly address uncertainty propagation. One approach, common in other RAG (Retrieval-Augmented Generation) systems, is to use an **external reranker module**.

- This module re-evaluates the chunks retrieved based on relevance rather than just vector similarity scores.
 - It uses another LLM to **rerank retrieved chunks**, reducing the amplification of bias or uncertainty in downstream synthesis.
Mendis itself doesn't use this reranker, but similar systems do.
-

Q3. Shouldn't a reranker ideally be a standard part of retrieval systems to reduce uncertainty?

A: In theory, yes. Some systems place the reranker between retrieval and synthesis to refine the top retrieved chunks (e.g., reranking the top 10 to pick the best 5).

The reranker can be a **separate model**, often fine-tuned to understand query relevance better than raw similarity metrics.

Q4. How is a reranker typically trained?

A: Usually, rerankers are **trained independently** of the main LLM, often using labeled datasets for relevance. They use the query and retrieved chunks as inputs, scoring how relevant each chunk is.

However, this adds computational overhead, since it's another LLM invocation on top of the existing pipeline.

Q5. The paper’s architecture discusses GPU and host memory — do they consider hierarchical or multi-GPU memory systems?

A: No. Their setup assumes a **single GPU model connected via PCIe** to a host system. They don’t discuss **hierarchical storage, multi-GPU setups**, or data sharding strategies. This is a major limitation since scaling to multiple GPUs would require handling inter-GPU communication and dataset partitioning.

Q6. Would larger GPU memory (e.g., 80GB or 96GB) make this problem disappear?

A: Not entirely. Larger memory alleviates but doesn’t eliminate the bottleneck. Even with 96GB GPUs, data transfer latency and memory management remain relevant issues—especially for multi-GPU clusters or dynamic workloads.

Q7. How generalizable is their approximate nearest neighbor (ANN) search approach?

A: It’s fairly general. Most vector retrieval systems now use **approximate KNN** for speed and scalability. Correctness doesn’t significantly change, though batch latency or query ordering can vary depending on how clusters are scheduled.

Q8. Could reinforcement learning improve query grouping or clustering?

A: Possibly, but it would be **computationally expensive**. The current method is a simple **throughput-oriented scheduling approach**, whereas reinforcement learning would introduce complex state and reward modeling that may not yield proportional benefits.

Q9. How should memory be designed for multi-agent systems where agents perform long workflows?

A: Memory systems should distinguish between:

- **Short-term (working) memory:** ephemeral, task-specific data.
- **Long-term memory:** persistent knowledge across sessions or agents.

Agents could benefit from a **logical routing layer** that directs queries to the appropriate vector database depending on task context, maintaining isolation between agents' memory spaces.

Q10. Should users or systems control what is remembered or forgotten?

A: Ideally, users should have **visibility and control** over memory—deciding what to retain or discard.

Current LLMs (like GPTs) often mix unrelated memories, bringing irrelevant context into new conversations.

Logical separation and user-facing memory management could improve relevance and privacy.

Q11. What are the system-level challenges in scaling memory and state management?

A: Even with better hardware, managing:

- **data locality,**
- **cache efficiency,** and
- **state consistency**

remains difficult.

The fundamental challenge isn't correctness but **building scalable, exciting architectures** that handle multi-agent memory efficiently and coherently.

Q12. Could caching retrieved chunks improve system efficiency?

A: Yes. One promising direction is **prefix caching** or **retrieval caching**:

- Cache commonly used chunks to avoid recomputation across similar prompts.

- Some systems already do local caching of recently retrieved clusters before re-querying global memory.
-

Q13. Beyond efficiency, what are open system research questions?

A:

- How to handle **tool integration** efficiently when agents call external APIs or functions.
 - How to maintain progress without blocking (e.g., running other tasks while awaiting tool responses).
 - How to handle **multi-modal and multi-agent coordination**, especially when context must be shared across modalities (text, image, actions) or between agents.
-

Q14. Could multi-agent interactions lead to race conditions in shared memory?

A: Potentially yes. If multiple agents write to shared memory simultaneously, it can lead to **race conditions** similar to those in classical distributed systems.

Exploring such **read/write conflicts** could open new research directions for consistency, synchronization, and conflict resolution in agentic memory systems.

Q15. What might be a “cool” or exciting problem in this domain?

A:

- Designing **cache-aware multi-agent memory systems** that balance speed, isolation, and consistency.
- Exploring **race conditions and concurrent memory edits** as new system-level challenges.
- Building systems that handle **cross-modal retrieval** (text + image + actions) with unified memory and context reasoning.